Constraint Satisfaction Problems Study Notes

Part 7: Local Search and Min-Conflicts

Learning Goal

By the end of this part, you should understand how some CSPs can be solved not by building a solution incrementally with backtracking, but by starting with a complete assignment and then repairing it. You should be able to explain local search, understand the min-conflicts heuristic, and see clearly when this approach is powerful and when it has limitations.

1. A Different Way to Think About Solving CSPs

Up to now, most methods have followed this pattern:

- start with no assignments,
- assign one variable at a time,
- backtrack when inconsistency appears.

Local search uses a very different idea.

Instead of building the assignment gradually, it begins with a **complete assignment**, meaning every variable already has a value. That assignment may violate constraints. The task is then to repeatedly change values in order to reduce the number of conflicts.

So local search does not ask, “How do I extend a partial solution?” It asks, “How do I repair a flawed complete solution?”

> **Things to Note:**
>
> - Backtracking works with partial assignments.
> - Local search works with complete assignments that may be inconsistent.
> - The search space in local search is the set of complete assignments, not partial ones.

2. What Local Search Means in CSPs

In a CSP, a **state** in local search is usually a full assignment of values to all variables. Some of those assignments satisfy all constraints; others violate some.

Local search moves from one complete assignment to another by making small changes. Usually, this means changing the value of one variable at a time.

The quality of a state is measured by the number of violated constraints, often called the number of **conflicts**.

Main objective

Find a complete assignment with zero conflicts.

Idea	Meaning in Local Search for CSPs
State	A complete assignment of values to all variables
Move	Change the value of one variable
Evaluation	Count how many constraints are violated
Goal	Reach a complete assignment with zero violated constraints

3. Why Local Search Can Be Attractive

Local search can be surprisingly effective because it avoids the heavy overhead of maintaining a large search tree. It often performs well on large CSPs where complete assignments are easy to generate and conflicts can be repaired quickly.

It is especially useful when:

- we only need one valid solution,
- the CSP is large,
- complete assignments are easy to construct,
- repairing conflicts is easier than exploring a huge backtracking tree.

> **Main Insight:**
>
> Local search trades systematic search for iterative improvement. It is less about proving correctness step by step and more about moving toward fewer conflicts.

4. Map Coloring Example: Complete Assignment with Conflicts

Consider a map-coloring CSP with variables `WA, NT, SA, Q, NSW, V, T`.

Suppose we start with this complete assignment:

- `WA = red`
- `NT = red`
- `SA = blue`
- `Q = green`
- `NSW = green`

- `V = red`
- `T = blue`

This is complete because every region has a color. But it is not necessarily legal.

For example:

- `WA` and `NT` conflict because they both have red.
- `Q` and `NSW` conflict because they both have green.

Local search would not back up and start a tree. Instead, it would choose a conflicted variable and try changing its color to reduce the number of violations.

5. Exam Scheduling Example: Complete Assignment with Conflicts

Now use the exam scheduling CSP with variables `CS101, CS102, MATH201, ENG101` and domain `{Mon-AM, Mon-PM, Tue-AM}`.

Suppose the constraints include:

- `CS101 ≠ CS102`
- `CS101 ≠ MATH201`
- `CS102 ≠ ENG101`
- `ENG101 ≠ Mon-AM`

Start with this complete assignment:

- `CS101 = Mon-AM`
- `CS102 = Mon-AM`
- `MATH201 = Tue-AM`
- `ENG101 = Mon-AM`

This assignment is complete, but it contains conflicts:

- `CS101` conflicts with `CS102`.
- `ENG101` violates the unary rule `ENG101 ≠ Mon-AM`.
- `CS102` conflicts with `ENG101` because they are in the same slot.

Again, local search repairs the assignment instead of discarding it completely.

> **High-Yield Point:**
>
> In local search, being wrong is acceptable at the start. The search begins from an imperfect complete assignment and improves it.

6. The Min-Conflicts Heuristic

The most important local-search method for CSPs in this topic is **min-conflicts**.

The idea is simple:

1. Start with a complete assignment.
2. If there are no conflicts, stop.
3. Choose a variable that is currently involved in a conflict.
4. Assign it the value that minimizes the number of conflicts.
5. Repeat until a solution is found or a stopping condition is reached.

This is called “min-conflicts” because at each step, the algorithm chooses the repair that creates the fewest violations.

Key feature

The algorithm does not necessarily make the assignment perfect immediately. It just tries to move in the direction of fewer conflicts.

7. Min-Conflicts in Map Coloring

Return to the map-coloring assignment:

- `WA = red`
- `NT = red`
- `SA = blue`
- `Q = green`
- `NSW = green`
- `V = red`
- `T = blue`

Suppose we choose the conflicted variable `NT`.

We test its possible values:

- If `NT = red`, it conflicts with `WA`.
- If `NT = green`, check conflicts with its neighbors.
- If `NT = blue`, check conflicts with its neighbors.

Whichever color creates the fewest conflicts is chosen.

For example, if changing `NT` from red to green removes its conflict with `WA` and does not create worse conflicts elsewhere, then the algorithm sets `NT = green`.

Then it looks again at the updated assignment and repeats the process.

8. Min-Conflicts in Exam Scheduling

Return to the exam-scheduling assignment:

- `CS101 = Mon-AM`
- `CS102 = Mon-AM`
- `MATH201 = Tue-AM`
- `ENG101 = Mon-AM`

Suppose we pick the conflicted variable `ENG101`.

Test the possible values:

- `ENG101 = Mon-AM` violates the unary constraint and may conflict with `CS102`.
- `ENG101 = Mon-PM` satisfies the unary rule and may reduce scheduling conflict.
- `ENG101 = Tue-AM` also satisfies the unary rule, but we must check for binary conflicts.

If `Mon-PM` produces the fewest total violations, then min-conflicts changes `ENG101` to `Mon-PM`.

Then the algorithm continues by selecting another conflicted variable, such as `CS102`, and again choosing the least-conflicting value.

Variable Chosen	Candidate Value	Effect
`ENG101`	`Mon-AM`	Keeps unary violation and may keep binary conflict
`ENG101`	`Mon-PM`	Removes unary violation and may reduce binary conflict
`ENG101`	`Tue-AM`	Removes unary violation but must be checked against other constraints

Things to Think About:

- Why does min-conflicts focus only on conflicted variables?
- Why is a complete but flawed assignment still useful?
- Why can small local repairs sometimes find a global solution quickly?

9. Why Min-Conflicts Often Works Well

Min-conflicts works well in many practical CSPs because a complete assignment often lies near a valid one. If only a few variables are causing trouble, changing them intelligently can quickly repair the solution.

This is especially effective when:

- constraints are numerous but not too rigid,

- a valid solution is surrounded by many almost-valid assignments,
- conflicts can be measured and repaired locally.

The method is famous for solving some large CSPs very efficiently.

10. Limitations of Local Search

Local search is powerful, but it is not perfect.

Its main weakness is that it can become stuck in a **local minimum**, a state where any small change seems worse even though the current state is not a solution.

It can also wander on plateaus where many states have similar conflict counts.

Map coloring intuition

You may reach a coloring where changing any one region alone does not seem to reduce the total number of conflicts, even though a better solution exists if two or more coordinated changes are made.

Exam scheduling intuition

You may reach a schedule where changing one exam's time simply shifts the conflict to a different course, so no single move seems helpful even though the overall schedule is not yet valid.

> **Common Confusion:**
>
> Min-conflicts is not guaranteed to systematically explore all possibilities like backtracking. It is a heuristic repair method, so it may fail or need restarting even when a solution exists.

11. Random Restarts and Practical Robustness

To deal with local minima, local search is often combined with **random restarts**.

This means:

- start from one random complete assignment,
- run min-conflicts for a while,
- if it gets stuck, generate a new random complete assignment and try again.

This improves robustness because a bad starting point does not doom the entire search.

Map coloring example

If one starting coloring leads to repeated cycling or stagnation, a different random coloring may quickly move toward a valid map.

Exam scheduling example

If one initial schedule traps the algorithm in repeated swaps among a few exams, a new random schedule may place the search in a much easier region of the space.

12. Backtracking vs Local Search

Feature	Backtracking Search	Local Search / Min-Conflicts
State type	Partial assignments	Complete assignments
Main method	Extend assignment and backtrack on failure	Repair conflicts iteratively
Systematic?	Yes	No, heuristic-based
Guarantee of completeness	Yes, with proper implementation	No such guarantee in basic form
Often strong for	Exact systematic solving	Large repair-friendly CSPs

These methods are not enemies. They reflect different philosophies of solving CSPs.

13. What the Examiner Usually Wants You to Understand

- That local search uses complete assignments rather than partial ones.
- That the objective is to reduce the number of conflicts until zero is reached.
- That min-conflicts chooses a conflicted variable and gives it the least-conflicting value.
- That local search can be extremely efficient on some CSPs.
- That it is heuristic rather than fully systematic, so it may get stuck.

> **Things to Note:**
>
> - Local search is often about repair, not construction.
> - Min-conflicts is one of the most important CSP local-search heuristics.
> - Being complete but inconsistent is acceptable at the start.
> - The search ends when conflict count becomes zero.

14. Quick Recap

- Local search starts with a complete assignment, even if it violates constraints.
- Each move changes one variable’s value in order to reduce conflict.

- Min-conflicts chooses a conflicted variable and assigns the value that creates the fewest violations.
- This method can work very well on large CSPs.
- Its weakness is that it may get stuck in local minima or plateaus.
- Random restarts can help overcome bad starting points.

15. Part 7 Revision Lines

- **Local search:** search among complete assignments.
- **Conflict:** a violated constraint.
- **Min-conflicts:** choose a conflicted variable and assign its least-conflicting value.
- **Main strength:** often very efficient on large CSPs.
- **Main weakness:** can get stuck, so restarts may be needed.

End of Part 7